

README

Nezha heavy-testing iteration 2 — QA evidence

Revision: 1 **Last modified:** 2026-06-16T13:34:01Z

Live stack: server-TLS <https://nezha.local:18443>, api-server <http://nezha.local:18080>, grpc nezha.local:50061, monitor :18090. Image rebuilt to f3904ccd71c2.

Finding #1 — verifier REST routes 404 on live server

Root cause (FACT): `cmd/server/main.go:140` registered the verifier routes only inside `if cfg.LLMsVerifier.Enabled`. The default-config nezha deploy has `Enabled=false`, so `/api/v1/verified-models`, `/verification-status`, `/translate-with-verification` (HTQ-API-003/004/006) were never added to the router → 404 (falsely "API does not exist").

RED baseline on live stack (pre-fix):

```
verified-models: 404
verification-status: 404
translate-with-verification: 404
```

Fix (safe, reversible — §11.4.101): ALWAYS register the routes; a `requireEnabled` gin guard returns an honest 503 `{"reason":"llmsverifier_disabled"}` when off (never 404), real upstream data when on. `pkg/api/verifier_handlers.go` (`enabled` flag + `SetEnabled` + `requireEnabled`) + `cmd/server/main.go` (unconditional registration).

Operator-policy note (iteration-3): a LIVE LLMsVerifier runs on nezha (`llmsverifier_llm-verifier_1`, 127.0.0.1:8080, `/api/models` 200) but on a DIFFERENT podman network (10.89.9.x) than `helixtranslate` translator-network. Fully enabling the real upstream needs an operator network-wiring + APIURL decision (blast radius). Safe choice taken: ship honest-503 always-on routes now.

Guard: `pkg/api/verifier_disabled_routes_test.go` — disabled routes 503-not-404 (mutation-proven: removing the guard → test FAILs).

Finding #2 — api-server /api/v1/health 503 IDLE false-negative

Root cause (FACT): gRPC `grpc.NewClient` conns are LAZY (start IDLE, dial on first RPC). `healthCheck` read `conn.GetState()` once → on a fresh boot saw IDLE → 503 "gRPC backend not ready" even though the backend was fully reachable (/providers 200 against the same backend, which had triggered an RPC).

Fix: `probeBackendState` actively kicks the lazy dial (`conn.Connect()`) and waits (bounded `healthProbeTimeout=2s`, `WaitForStateChange`) for READY before deciding. IDLE-against-reachable → READY (healthy); IDLE-against-unreachable → TRANSIENT_FAILURE (still unhealthy — never blindly trusts IDLE). `cmd/api-server/main.go`.

Guard: `cmd/api-server/`

`health_test.go::TestHealthCheck_LazyIdleButReachableBackend_ReportHealthy` (real in-process gRPC backend; mutation-proven: reverting to one-shot `GetState` → 503 IDLE → test FAILs). Existing not-Ready→503 guards retained.

Finding #3 — compose \${VAR} interpolation reads .env not .env.nezha

Root cause (FACT): `podman-compose` resolves `${VAR:-default}` from its DEFAULT `dotenv (.env) + shell env`, NOT from `env_file:` (which only injects INTO the container). `nezha .env` holds only `HELIX_RELEASE_PREFIX`; `*_HOST_PORT + JWT_SECRET` live in `.env.nezha` → fell back to defaults (50051/8080/8443/8090, `change-me-in-env-nezha`) unless the operator manually set `-a; . .env.nezha`.

Fix (durable): `scripts/nezha-deploy.sh` always invokes `podman-compose --env-file .env.nezha`. Proven in a CLEAN shell (no export): `bash scripts/nezha-deploy.sh config` resolves real 18080/18443/50061/18090 + real JWT (no `change-me` fallback).

Also fixed: `reboot` action does explicit `build → stop → rm -f → up` because `podman-compose 1.5.0 up --build` (and even `--force-recreate`) does NOT replace a running container when the image tag is unchanged (§11.4.108 gap — the rebuilt image existed but live containers kept the old binary).

Guard: `scripts/nezha_deploy_envfile_guard_test.sh` (RED_MODE polarity: a no-`--env-file` invocation is flagged).

Operator note: `.env.nezha` line 48 is malformed (a space-separated list of provider-key NAMES with one =value at the end); `python-dotenv` cannot parse it, so those provider keys are not injected. Operator-owned secrets file — flagged, not edited (§11.4.10).

Finding #4 — gRPC real translate round-trip

Deferred to iteration-3: gRPC translate is an async FILE-job model (StartTranslation(input_file,output_file,...) → poll GetTranslationStatus → read generated files). A real round-trip needs an input file staged into the grpc-server container + a working provider key (blocked by the #3 line-48 key defect). Scoped for iter-3 with a small Go client.